

Additive Baselines on the 025 Build

The trigenic ladder under a random and a query-pair-disjoint split, what each model sees, and what the graphs did

Michael Volk

September 15, 2026

Abstract

A model that claims to learn genetic interaction has to beat a null that cannot represent interaction. Five such nulls, and a nonlinear baseline on the same features that can, are fit on exactly the 376,732 trigenic records and the two splits the 025 transformer arms train on. Under the published random-over-records split the per-gene additive ridge reaches 0.400 test Pearson, a model that sees only which Kuzmin query double a triple carries reaches 0.390, and the transformer checkpoints reach 0.438 to 0.455. Reading the model code establishes what that comparison is between: the transformer’s encoder never sees the strain, the nine interaction graphs enter training through a penalty on layer-1 attention rather than through the function that maps a checkpoint to a prediction, and the strain-dependent computation is a set function over the three perturbed gene identities, the same input and the same hypothesis-class shape the nulls have. The penalty was not incidental: it carried 99.99 percent of the 010 training loss, the regularized heads recover their graphs at 0.73 to 1.00 recall, and a matched pair of runs without it fails to train. Rebuilding the forward pass reproduces the recorded metrics to the fourth decimal and shows the transformer is not a relabeled additive model, correlating with it at 0.73 to 0.76 rather than the 0.999 reported for MULTI-evolve, with a gain of +0.038 to +0.055 Pearson whose bootstrap intervals exclude zero.

The result that changes what the metric means is the split. Holding out whole query doubles collapses the additive ridge to 0.185 on the one disjoint split the transformer trains on, and to 0.127 ± 0.033 over five disjoint folds, and drops the nonlinear baseline below the additive one. Four transformer runs have trained on the disjoint split, one in the 010 configuration and three with a fixed sequence embedding and a constant learning rate. Every one peaks on validation between epoch 2 and epoch 7, at 0.199 to 0.270 against an additive validation null of 0.150, and every one falls back afterward; the 010 configuration ends below the null at 0.131. Its validation-selected checkpoint, scored on the 46 held-out test screens, reaches 0.139 against the additive ridge’s 0.185, a paired gap of -0.046 with a bootstrap interval of $[-0.056, -0.036]$, and is level with the nonlinear baseline. Under the prespecified criteria the transformer in the 010 configuration is not shown to beat the additive null on held-out screens, and the one scored checkpoint is below it; the seed replicates that would make this a statement about the model rather than about one run have not been run. Every run is in a registry linked to its W&B page, and three earlier claims are corrected in place.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Provenance: ▲ not in the library mirror; verify at the linked source ● from a review’s summary table, not the primary paper

Contents

1	The question, and the state of the evidence ✗	3
2	Data and the two splits ✗	3
3	The models ✗	4
3.1	Where each baseline comes from ✗	4
3.2	Notation and the interaction operator ✗	5
3.3	B0, the train mean ✗	5
3.4	B1, the additive per-gene ridge ✗	5
3.5	B2, additive plus recurring gene pairs ✗	5
3.6	B3, the empirical screen mean ✗	6
3.7	B4, the screen indicator ✗	6
3.8	B5, nonlinearity on the same features ✗	6
3.9	The transformer ✗	6
3.10	Summary of capacity ✗	7
4	What each model actually sees ✗	7
4.1	The transformer’s per-sample input ✗	7
4.2	The encoder does not depend on the strain ✗	7
4.3	Where the nine graphs act, and where they do not ✗	8
4.4	Is the comparison fair ✗	8

5	The ladder under both splits	11
5.1	Reading the ladder	11
5.2	Verifying the architecture reading	14
5.3	What the graphs did during training	14
5.4	Record-by-record agreement	15
6	The transformer on held-out screens	16
6.1	The run registry	16
6.2	Validation, then the one test score	16
6.3	The prespecified decision	18
6.4	Corrections recorded	18
7	What is established, and what is left	19
7.1	Established	19
7.2	Not established	19
7.3	What is left before this enters the main text	19

1 The question, and the state of the evidence ✗

Tran et al. introduced MULTI-evolve in *Science* in 2026 (10.1126/science.aea1820), a protein engineering framework whose fully connected networks were said to learn the epistatic landscape from single and double mutants and to identify synergistic combinations at higher order. Visani, Verma and DeWitt replied in a preprint, *Additive baselines furnish no evidence for epistasis learning by MULTI-evolve* (bioRxiv 2026.04.23.719915), which does one thing: it fits the null that the original paper never fit. An **additive model** over mutations predicts a phenotype as a sum of per-mutation terms and assigns exactly zero to every interaction, so it is the natural null for any claim that a model has learned interaction, as an intercept-only model is the null for a claim that a model has learned anything. Their networks' predictions correlated with a ridge-regularized additive model's at $r > 0.999$, and on held-out data the network did not significantly outperform the additive model or a zero-hidden-layer version of itself.

The analogous question for the 010 trigenic transformer is whether its held-out performance on the trigenic interaction score survives the same test, and it has a second part the protein setting does not have: the published split lets every model score the identity of the screen a record came from. Both are answered here on the 025 build, which holds the 010 records with bit-identical labels, under two splits: the published random split (arm R) and a split that never shares a Kuzmin query double between parts (arm Q).

One difference in the setup, and it cuts against the null. In the protein setting the quantity predicted is the multimutant phenotype itself, which per-mutation effects explain directly. Here the quantity predicted is already a residual: the trigenic interaction score subtracts the expectation built from single and double mutant fitness before the model ever sees it. An additive-in-single-gene-effects model is therefore a harder null here than it is there, and its performance is more surprising, not less. That argument has one loophole, the screen structure of Section 2, and quantifying it turns out to matter more than anything else below.

Measured. Five nulls and the nonlinear baseline on both arms, on test (Table 7, Figure 3a,b). Arm R reproduces the 010 numbers. On arm Q the additive ridge falls from 0.400 to 0.185, the screen-only model to zero, and the nonlinear baseline from 0.432 to 0.141, below the ridge. Four transformer runs on arm Q, every one peaking on validation by epoch 7 and declining afterward (Figure 5a,b). One transformer test score on arm Q: GH 1640's validation-selected checkpoint at 0.139, below the ridge by 0.046 with a paired interval that excludes zero, level with the nonlinear baseline (Table 15).

Not measured. Replicates of the 010 configuration on arm Q, which is what turns one checkpoint's number into a statement about the model, and the learnable-table control for the constant-learning-rate runs.

Vocabulary. **Arm R** and **arm Q** are the two training arms of experiment 025, one per split; every baseline is fit under both. The **ladder** is the ordered set of baselines B0 through B5 and the transformer, read left to right by interaction capacity. A **query pair** is the double mutant a Kuzmin trigenic screen was built around; every record in the build is one query pair plus one **array gene**, and 420 query pairs recur across the 376,732 records. A **screen** is the set of records sharing a query pair. A **validation maximum** is the largest validation Pearson over a run's logged epochs; it is an upward-biased order statistic, and it is never set beside a test number without that label.

2 Data and the two splits ✗

The label. Write f_i , f_{ij} and f_{ijk} for the fitness of the single, double and triple mutant relative to wild type, and $\varepsilon_{ij} = f_{ij} - f_i f_j$ for the digenic interaction. The trigenic interaction score is

$$\tau_{ijk} = f_{ijk} - f_i f_j f_k - f_i \varepsilon_{jk} - f_j \varepsilon_{ik} - f_k \varepsilon_{ij}, \quad (1)$$

▲ [external: Kuzmin et al. 2018, 10.1126/science.aao1729; the definition of the released adjusted score, not re-derived here] a third-order residual: the multiplicative part of fitness and every pairwise term are subtracted before the number is reported, so τ is by construction the part of the triple phenotype that no single or double mutant predicts. The loaders store it as `gene_interaction`.

The records. Subset S0 of the 025 build is its 376,732 trigenic records over 4,352 genes. Matching them to the 010 build by genotype recovers every one; 376,526 labels are bit-identical and the remaining 206 differ by at most 5.6×10^{-17} (`experiments/025-solid-growth/scripts/label_parity_010_vs_025.py`). Every model below is fit on these records and nothing else.

The screen structure. The records contain 739,315 distinct unordered gene pairs, of which 420 occur five or more times. Those 420 account for 376,733 pair instances, one more than the record count: every record contains exactly one recurring pair, and a single record contains two. The 420 pairs are the Kuzmin query doubles. Grouping records by

their recurring pair recovers the screen exactly, and any model with a per-pair offset can absorb whatever is common to a screen.

The two arms. Arm R is 010’s own split, random over records, carried into the 025 index space by genotype identity. Arm Q assigns whole query pairs to parts, so no screen appears in more than one part. The two arms hold out nearly the same number of records and differ only in what is held out.

Table 1: The two arms. Both partition the same 376,732 records. Each part of arm R carries every one of the 420 query pairs; the parts of arm Q carry disjoint sets of them. The split artifacts are the files the trainer loads, under `experiments/025-solid-growth/results/`.

	Arm R	Arm Q
Held-out unit	record	query pair
Recurring query pairs	420, shared by every part	420, no pair in two parts
Query pairs, train / val / test		331 / 43 / 46
Records, train / val / test	301,386 / 37,673 / 37,673	301,236 / 37,705 / 37,791
Split artifact	<code>pinned_splits_from_010_seed_42.json.gz</code>	<code>query_pair_disjoint_splits_025.json.gz</code>
Transformer run	GH 1598	GH 1640, and Table 13

What arm Q holds out. The query pair, not the genes. Every array gene of the validation and test parts occurs in training, as do 78 of the 86 validation and 88 of the 92 test query-pair genes, so 81 percent of validation records and 91 percent of test records have all three genes in the training set (Table 2). A model on arm Q is therefore asked to predict an unseen pair of mostly seen genes with a seen third gene, which is the generalization that scoring an unmeasured triple requires. The test part spreads its records over 1,182 distinct array genes against the validation part’s 4,003; the two parts are different kinds of screen sample, and no analysis here uses that difference.

Table 2: What the held-out parts of arm Q share with its training part. A query pair is the most frequent recurring pair a record carries, the rule that built the split; the array gene is the record’s third gene.

	Validation	Test
Records	37,705	37,791
Query pairs held out	43	46
Query pairs also in training	0	0
Distinct array genes	4,003	1,182
Array genes present in training	4003 of 4003	1182 of 1182
Query-pair genes present in training	78 of 86	88 of 92
Records with all three genes in training	81.3%	91.4%

3 The models ✗

3.1 Where each baseline comes from ✗

Two of the six baselines are taken from the Visani preprint. The rest were built here for a confounder that the protein setting does not have. Naming which is which matters, because a baseline borrowed from a published critique carries that critique’s authority and one invented here does not.

Table 3: Provenance of the six baselines.

Model	Origin	Corresponds to
B0	standard	intercept-only null
B1	Visani et al. 2026	their ridge-regularized additive model
B2	this analysis	B1 extended to observed pairs
B3	this analysis	unregularized counterpart of B4
B4	this analysis	the Kuzmin screen-structure confounder
B5	Visani et al. 2026	their zero-hidden-layer control, run in reverse

B1 is their additive null transplanted onto gene deletions. B5 inverts their zero-hidden-layer control: where they removed nonlinearity from the published architecture, B5 adds the smallest amount of nonlinearity to the additive feature space, so the two bracket the same gap from opposite sides. B3 and B4 have no counterpart in the preprint because the confounder they measure is specific to a synthetic genetic array: a protein engineering campaign has no analog of a query double mutant recurring across thousands of measured variants.

3.2 Notation and the interaction operator $\times$

Let G be the 4,352 genes appearing in the build and let record n have perturbed set $S_n \subset G$ with $|S_n| = 3$. Encode it as $x_n \in \{0, 1\}^{|G|}$ with $x_{n,g} = 1$ exactly when $g \in S_n$, so that $\|x_n\|_1 = 3$ for every record, and write y_n for the trigenic interaction score of Eq. 1.

A model is a set function $f : 2^G \rightarrow \mathbb{R}$. Its capacity to represent interaction is measured by finite differences. For distinct genes g, h, k and any S disjoint from them,

$$\Delta_g f(S) = f(S \cup \{g\}) - f(S), \quad (2)$$

$$\Delta_{gh} f(S) = f(S \cup \{g, h\}) - f(S \cup \{g\}) - f(S \cup \{h\}) + f(S), \quad (3)$$

$$\Delta_{ghk} f(S) = \sum_{U \subseteq \{g, h, k\}} (-1)^{3-|U|} f(S \cup U). \quad (4)$$

$\Delta_{gh} f \equiv 0$ says the model cannot express any pairwise interaction; $\Delta_{ghk} f \equiv 0$ says it cannot express any three-way interaction. These are the properties that make a model a null, and they are structural, holding for every parameter setting rather than for a fitted one.

3.3 B0, the train mean $\times$

$\hat{y} = \bar{y}_{\text{train}}$. Every finite difference vanishes. Its mean squared error is the label variance, and it fixes the zero point against which all reductions are read.

3.4 B1, the additive per-gene ridge $\times$

$$\hat{y}(S) = \beta_0 + \sum_{g \in S} \beta_g, \quad \hat{\beta} = \arg \min_{\beta} \sum_{n \in \text{train}} (y_n - \beta_0 - x_n^\top \beta)^2 + \alpha \|\beta\|_2^2, \quad (5)$$

with β_0 unpenalized and set to $\bar{y}_{\text{train}}$. Writing X for the $n_{\text{train}} \times |G|$ matrix whose rows are the x_n and $\tilde{y} = y - \bar{y}_{\text{train}}$ for the centered labels, the minimizer is the ridge normal equation

$$\hat{\beta} = (X^\top X + \alpha I)^{-1} X^\top \tilde{y}, \quad (6)$$

which the script evaluates by LSQR on the augmented system $[X; \sqrt{\alpha} I] \beta = [\tilde{y}; 0]$ rather than by forming the inverse. Substituting Eq. 5 into Eq. 3 gives $\Delta_{gh} \hat{y} = 0$ and hence $\Delta_{ghk} \hat{y} = 0$ identically. B1 is the direct analog of the additive null in the Visani preprint.

The design is well conditioned despite having 4,352 coefficients: with three ones per row and 301,386 training rows there are 904,158 gene observations, a median of 9 per gene in the training split with a maximum of 3,430. The ridge penalty is chosen on validation over $\alpha \in \{10^{-2}, 10^{-1}, 1, 3, 10, 30, 100, 300, 1000\}$ and the selected value is then reported on test.

3.5 B2, additive plus recurring gene pairs $\times$

Let P be the set of unordered gene pairs occurring at least five times in the training split, $|P| = 420$. Adding one coefficient per such pair,

$$\hat{y}(S) = \beta_0 + \sum_{g \in S} \beta_g + \sum_{\{g, h\} \subseteq S, \{g, h\} \in P} \gamma_{gh}, \quad (7)$$

gives $\Delta_{gh} \hat{y} = \gamma_{gh}$ for a pair in P and zero otherwise. B2 therefore has second-order capacity on observed pairs. It still has *no* third-order capacity: $\Delta_{ghk} \hat{y} = 0$ identically. Since the label is itself a third-order residual, B2 remains a null with respect to the quantity being predicted.

The fit is the same ridge as B1 on the wider design $X = [X_G \ X_P]$, where X_P is the $n_{\text{train}} \times |P|$ pair-incidence matrix, so $(\hat{\beta}, \hat{\gamma})$ is Eq. 6 with X replaced by $[X_G \ X_P]$ and one penalty α on both blocks. The pair block has a closed form of its own that says what B2 learns. Every record contains exactly one recurring pair (Section 2), so the stationarity condition of the ridge objective in γ_{gh} involves only that pair's records, and

$$\hat{\gamma}_{gh} = \frac{\sum_{n \in \text{train}, \{g, h\} \subseteq S_n} (y_n - \beta_0 - x_n^\top \hat{\beta})}{n_{gh} + \alpha}, \quad (8)$$

with n_{gh} the number of training triples carrying the pair. Each pair coefficient is the mean residual of the additive model over that query double's screen, shrunk toward zero by $\alpha/(n_{gh} + \alpha)$. With n_{gh} in the hundreds to thousands and the selected α in the units to tens, the shrinkage is at most a few percent, so B2 is B1 plus one nearly unshrunk offset per screen.

3.6 B3, the empirical screen mean ✖

B3 has no fitted parameters. Its per-gene and per-pair quantities are means of the training label over the records that contain the gene or the pair,

$$\bar{y}_g = \frac{\sum_{n \in \text{train}} x_{n,g} y_n}{\sum_{n \in \text{train}} x_{n,g}}, \quad \bar{y}_{gh} = \frac{\sum_{n \in \text{train}} x_{n,g} x_{n,h} y_n}{\sum_{n \in \text{train}} x_{n,g} x_{n,h}}, \quad (9)$$

so $\bar{y}_g$ averages y over exactly the training triples in which gene g is one of the three deletions, and $\bar{y}_{gh}$ over those carrying both g and h . The prediction backs off through three levels,

$$\hat{y}(S) = \begin{cases} \text{mean}\{\bar{y}_{gh} : \{g, h\} \subseteq S, n_{gh} \geq 5\} & \text{if any pair in } S \text{ recurs,} \\ \text{mean}\{\bar{y}_g : g \in S, n_g \geq 5\} & \text{otherwise, if any gene in } S \text{ has 5 training records,} \\ \bar{y}_{\text{train}} & \text{otherwise,} \end{cases} \quad (10)$$

with n_g and n_{gh} the training counts. On the random split every test record carries a recurring pair, so the first case always fires and B3 is the per-screen mean of the label. On a query-pair-disjoint split no test record carries a recurring pair, the first case never fires, and B3 becomes the mean of its three gene means.

3.7 B4, the screen indicator ✖

B4 drops the per-gene terms from Eq. 7 and keeps only the pair terms,

$$\hat{y}(S) = \beta_0 + \sum_{\{g,h\} \subseteq S, \{g,h\} \in P} \gamma_{gh}, \quad \hat{\gamma}_{gh} = \frac{\sum_{n \in \text{train}, \{g,h\} \subseteq S_n} (y_n - \bar{y}_{\text{train}})}{n_{gh} + \alpha}, \quad (11)$$

where the second expression is Eq. 8 with no gene block. Because the recurring pairs are the Kuzmin query doubles, B4 is a per-screen offset: for a triple made of a query pair and an array gene, the array gene contributes nothing at all. Whatever B4 achieves is therefore an upper bound on how much of the metric is explained by knowing which screen a record came from.

Eq. 11 at $\alpha \rightarrow 0$ is $\bar{y}_{gh}$ of Eq. 9, so on the random split, where the first case of Eq. 10 always fires, B3 and B4 are the same predictor up to the shrinkage $\alpha/(n_{gh} + \alpha)$, which Pearson does not see. Table 7 reports them equal to three decimals for that reason. On a disjoint split they part company: B4 is structurally zero and B3 falls back to gene means.

3.8 B5, nonlinearity on the same features ✖

Give each gene a free vector $e_g \in \mathbb{R}^{128}$, sum over the perturbed set and read out with a two-hidden-layer network ϕ ,

$$\hat{y}(S) = \phi\left(\sum_{g \in S} e_g\right), \quad \phi : \mathbb{R}^{128} \rightarrow \mathbb{R}. \quad (12)$$

This is permutation invariant and uses precisely B1's feature space, the identity of the three genes and nothing more. Because ϕ is nonlinear, $\Delta_{gh}\hat{y}$ and $\Delta_{ghk}\hat{y}$ are not identically zero, so B5 is not a null. It is the smallest departure from B1 that has interaction capacity, and it isolates what capacity alone buys. It is trained by gradient descent with early stopping on validation, with three seeds, so it reports a spread rather than a single number.

3.9 The transformer ✖

The cell graph transformer of experiment 010, run on the 025 build with no change to architecture, loss, optimizer or schedule (cgt_s0_r_kl_000 for arm R, cgt_s0_q_kl_004 for arm Q). Gene identity enters through a free table E of 6,607 rows and 180 columns, which an eight-layer encoder T maps once, at batch size one and independently of the strain, to per-gene vectors $Z = T(E)$ and a CLS vector h_{CLS} . For a record with perturbed set S , the perturbation transform performs cross-attention in which all genes query the rows $\{Z_p : p \in S\}$, followed by a residual, a layer norm and a feedforward block, giving $H_i^{\text{pert}} = g(Z_i + c_S)$ with c_S the attended context. The head pools over the perturbed genes and reads out,

$$z_S = \frac{1}{|S|} \sum_{i \in S} H_i^{\text{pert}}, \quad \hat{y} = \text{MLP}([h_{\text{CLS}} \parallel z_S]), \quad (13)$$

with the MLP being $\text{Linear}(360, 180) \rightarrow \text{ReLU} \rightarrow \text{Linear}(180, 1)$. Since h_{CLS} is constant, Eq. 13 is a function of z_S alone, and z_S is a permutation-invariant pooling of quantities derived from $\{Z_p : p \in S\}$: the transformer is a set function on the three perturbed gene identities, built on a free per-gene vector, the same functional shape as the nulls, which are set functions on the same identities built on a free per-gene scalar. Comparing Eq. 12 with Eq. 13, both give every gene a free vector, pool over the three perturbed genes in a permutation-invariant way, and pass the pooled vector through a small network; the transformer’s pooling is richer and its per-gene vectors are $Z = T(E)$ rather than a raw table. The nine interaction graphs enter only through a penalty on layer-1 attention during training, so they shape the weights and are absent from the prediction function (Section 4).

3.10 Summary of capacity ✗

Table 4: What each model can represent. B3 is listed by the case of Eq. 10 that fires: the pair case on the random split, the gene case on a query-pair-disjoint split. The last two columns are structural properties of the hypothesis class, not fitted results. Only B5 and the cell graph transformer can express a three-way interaction, which is what the label is.

Model	Per-gene quantity	Set aggregation	Fit by	$\Delta_{gh} \neq 0$	$\Delta_{ghk} \neq 0$
B0	none	none	train mean	no	no
B1	free scalar β_g	sum	ridge, Eq. 6	no	no
B2	free scalar β_g	sum, plus γ_{gh} per recurring pair	ridge, Eq. 8	yes	no
B3, pair case	none	mean of $\bar{y}_{gh}$ over the recurring pairs in S	label means, Eq. 9	yes	no
B3, gene case	$\bar{y}_g$, the label mean over training triples containing g	mean over S	label means, Eq. 9	no	no
B4	none	γ_{gh} per recurring pair	ridge, Eq. 11	yes	no
B5	free 128-vector e_g	sum, then MLP	gradient descent	yes	yes
CGT	free 180-vector via $T(E)$	attention over S , mean, MLP, Eq. 13	gradient descent	yes	yes

Comparability. Every model is fit on the training part of the arm it is reported under, makes its free choices on the validation part, and is reported on test, over the full part in raw τ units. The transformer’s label normalizer is fit on the training part for arm Q and on all of S0 for arm R; Pearson is invariant to that choice, and the nulls use the training mean only (Section 4.4).

4 What each model actually sees ✗

The comparison is only meaningful if the models are given the same information. Reading the 010 code settles that, and the answer is stronger than expected: every model in the ladder, the transformer included, receives exactly one thing per record, the set of three perturbed genes.

4.1 The transformer’s per-sample input ✗

The dataset is built with the `Perturbation` graph processor. That processor writes the perturbed gene names, their indices, a boolean mask and the label into each sample, and nothing else (`graph_processor.py`, lines 1984 to 2058). There is no per-gene feature matrix: with an empty `node_embeddings` list in the config, the gene feature tensor of the cell graph has shape 6607×0 (`cell_data.py`, line 53). Gene identity is carried by a free lookup table of 6,607 rows and 180 columns (`equivariant_cell_graph_transformer.py`, line 1989), initialized randomly and fit by gradient descent. No sequence, expression, ontology or other precomputed representation enters. The model is given gene identities, the same as a one-hot design matrix, and learns its own 180-dimensional vector for each.

4.2 The encoder does not depend on the strain ✗

The forward pass builds its input as `gene_embedding(torch.arange(N))`, prepends a CLS token, and unsqueezes to shape $[1, N+1, 180]$. That leading dimension is 1, not the batch size. The eight transformer layers then run on this single tensor, which is assembled entirely from parameters and contains nothing from `batch`. Writing E for the embedding table and T for the eight-layer encoder,

$$Z = T(E) \in \mathbb{R}^{N \times 180}, \quad h_{\text{CLS}} \in \mathbb{R}^{180}, \quad (14)$$

and Z and h_{CLS} are the same for every record in the dataset. The repository states this itself, in the docstring of `PerturbationGraphPropagation`: “The encoder runs at batch 1 on the WILDTYPE graph, so `H_genes` and `h_CLS` are identical for every strain; the only strain-dependent step is `EquivariantPerturbationTransform`.”

So the eight encoder layers, which hold 3,129,120 of the model’s 4,774,861 parameters, perform no per-sample computation. They map one free parameter block to another. Whatever role they play, it is not to process the strain. What remains after Eq. 14 is the strain-dependent part, Eq. 13, and that is a set function on the three perturbed gene identities. The models in the ladder differ in how expressive the set function is, not in what they are allowed to see. Figure 1 draws the pass with the two entry points marked.

4.3 Where the nine graphs act, and where they do not ✗

The nine adjacency matrices enter the model in exactly one place. During training, a Kullback-Leibler penalty pulls one attention head of layer 1 toward each row-normalized adjacency $\tilde{A}^{(k)}$,

$$\mathcal{L}_{\text{graph}} = \sum_{k=1}^9 c_k \sum_{i=1}^N \sum_{j=1}^N \tilde{A}_{ij}^{(k)} \log \frac{\tilde{A}_{ij}^{(k)}}{\alpha_{ij}^{(k)}}, \quad \alpha^{(k)} = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right)_{h_k}, \quad (15)$$

and that is the only operation that reads them.

Not in the prediction. Three mechanisms could put adjacency into the computation that yields $\hat{y}$: a head mask on attention, the perturbation-propagation block, and an adjacency bias inside the attention logits. All three are off in the 010 configuration. Given a trained checkpoint, no adjacency tensor is an input to any operation between the gene indices and the prediction, and Section 5.2 checks this directly: setting the penalty weight to zero on a trained checkpoint moves the predictions by at most 1.3×10^{-6} .

In the weights. The penalty is computed inside the same forward call as the prediction, from the same pre-dropout attention tensor, and nothing detaches it. Its gradient reaches the layer-1 query and key projections, through them every layer-0 parameter, and through those the gene embedding table. So $Z = T(E)$ of Eq. 14, the per-gene vector the prediction is read from, is shaped by the nine graphs during training even though its value at inference comes off fixed weights. The graphs act on the parameters, not on the function from checkpoint to prediction. An ablation that zeroes the coefficient on a trained checkpoint tests only the second.

How large the penalty was. The per-graph weight c_k in Eq. 15 is a product of three factors: the per-head $\lambda_k = 10^{-3}$, a global scale of 1.0 read from `regression_task.loss.graph_regularization.lambda`, and an edge normalization meant to divide by the mean degree. That last factor is computed as `len(A.nonzero()[0])` summed over the nine matrices. On a dense `torch` tensor `A.nonzero()` has shape `[nnz, 2]`, so `[0]` is one index pair of length two, and the sum over nine graphs is 18 rather than the 2.5 million edges intended. Dividing by 18/6607 multiplies by 367.06,

$$c_k = 10^{-3} \times 1.0 \times \frac{6607}{18} = 0.367, \quad (16)$$

applied to a divergence summed over all 6,607 rows. The 010 runs’ logs record the consequence: against a point loss of 0.81 the graph term is 5,740, and the trainer’s diagnostic `norm_weighted_graph_reg`, the graph term divided by the total loss, reads 0.99986 for all three checkpoints. The optimizer spent 99.99 percent of its objective on the graph penalty. Figure 2 shows the three loss terms and that share over the whole of training, and Section 5.3 reports what the penalty bought.

The dominance was not what produced the accuracy. The 025 replication of this configuration, GH 1598, ran after the edge normalization had been corrected, so its graph term is 0.34 against a point loss of 0.80, a share of 0.29 to 0.33 rather than 0.9999, and it reached validation Pearson 0.446 at epoch 14 against 0.447 to 0.462 for the three 010 checkpoints.

All nine heads carried the penalty. The configuration names the first two graphs `physical` and `regulatory`, which is what the cell graph called them at the December 2025 commit these runs used; the suffixed names date from 2026-06-27. The runs log the edge-recovery keys `physical_L1_H0` and `regulatory_L1_H1`, which the trainer emits only for a graph it matched.

4.4 Is the comparison fair ✗

Six things have to match before a baseline number can sit in the same table as a transformer number. Table 5 names them and gives the verdict on each.

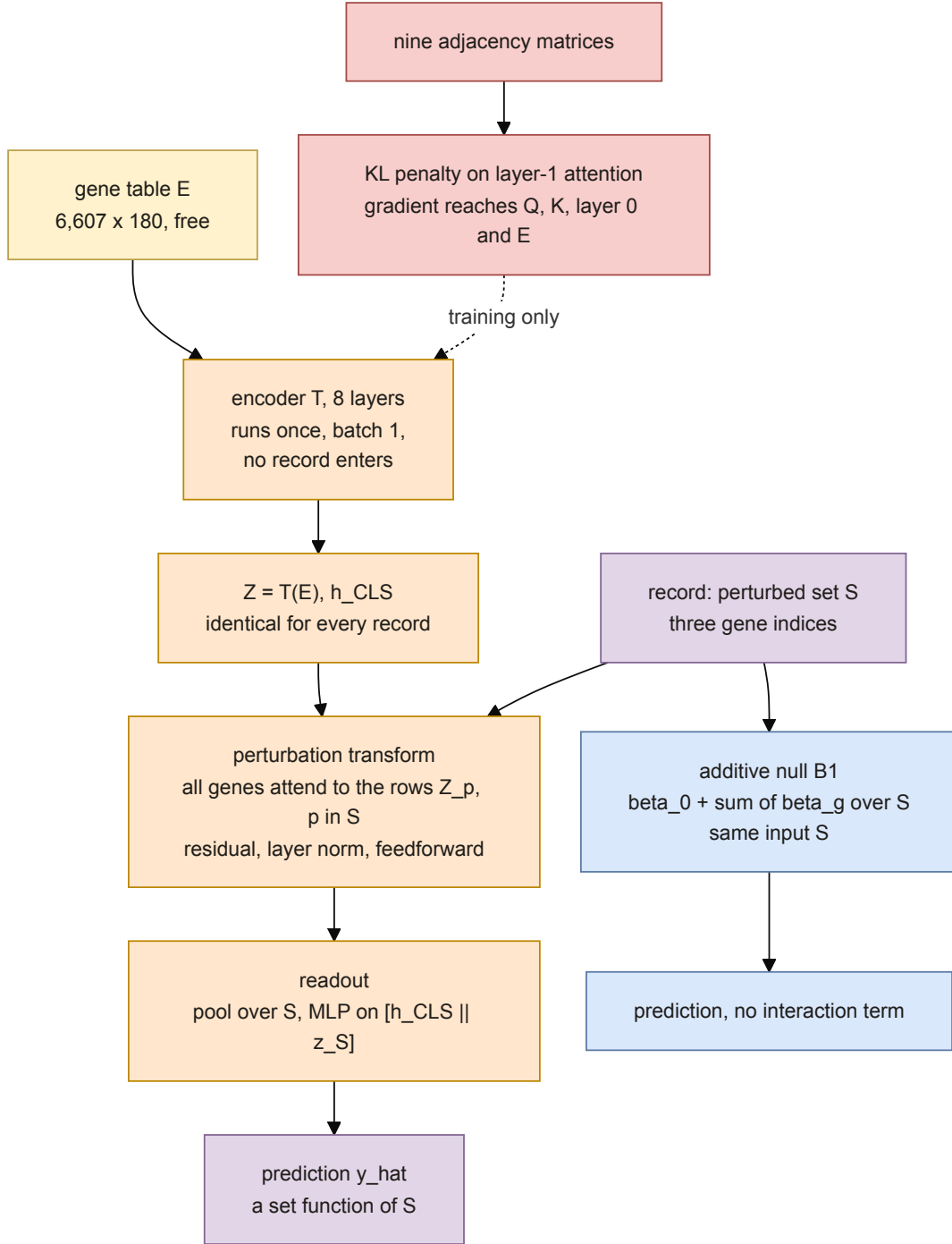


Figure 1: The forward pass of the 010 configuration, and the additive null beside it. Yellow is a parameter block, orange a computation, purple what a record contributes and what comes out, red what exists only during training, blue the null on the same input. The record enters once, as three gene indices, at the perturbation transform; the graphs enter once, through the layer-1 attention penalty, and reach the prediction only through the trained weights.

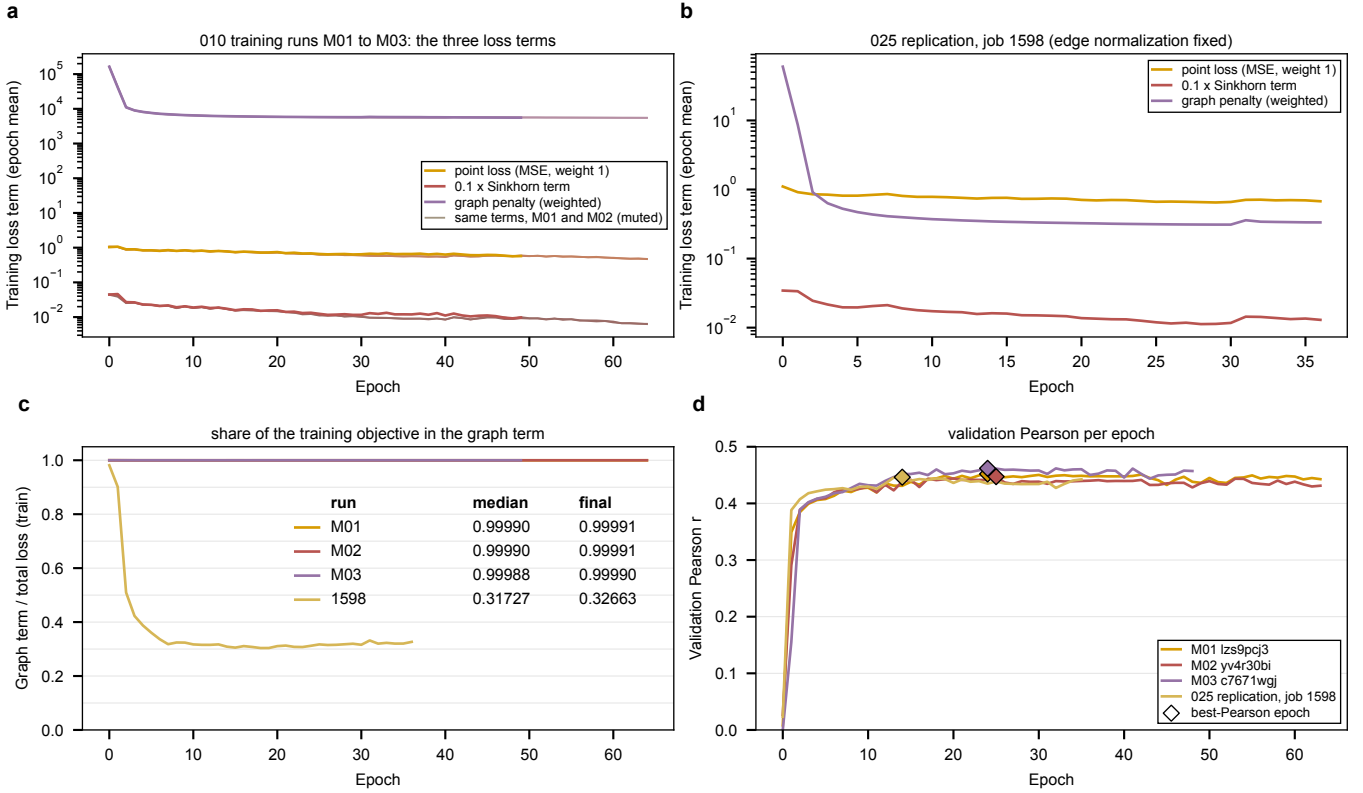


Figure 2: The training objective of the three 010 runs and of the 025 replication, per epoch. **a**, the point term, the Sinkhorn term at its weight of 0.1, and the weighted graph penalty for the 010 runs, on a log axis: the penalty sits four orders of magnitude above the point loss from epoch 5 on. **b**, the same three terms for GH 1598, where the corrected edge normalization puts the penalty below the point loss. **c**, the graph term’s share of the total loss, recomputed from the logged terms: 0.9999 throughout for the 010 runs, 0.32 for the replication. **d**, validation Pearson per epoch with the best-Pearson epoch marked; the replication reaches the 010 band with the penalty at a third of the objective.

Table 5: What has to match for the comparison to be fair. Row 6 is the one mismatch, and it holds for arm R only. It is a hygiene defect in the 010 configuration rather than a source of advantage, and arm Q fits its constants on the training part.

#	Verdict	What has to match
1	matches	Records and split. All models use the same 376,732 records and, per arm, the same partition (Table 1).
2	matches	Label. All predict <code>gene_interaction</code> , the published adjusted τ of Eq. 1.
3	matches	Reported split. Test is reported for every baseline row, and validation is what every row selected on. The transformer rows that are validation maxima say so in their label.
4	matches	Metric definition and denominator. Pearson, Spearman and mean squared error over the full held-out part in raw τ units.
5	matches	Information available. The identity of the three perturbed genes, and nothing else, for every model (Sections 4.2 and 3.9).
6	arm R only	Label normalization. The arm R transformer trains on z-scored labels whose mean and standard deviation were computed from all 376,732 records, so two held-out scalars touch a training-time constant; the baselines use the training mean only. Pearson is invariant to the two constants, so this cannot move the reported metric. Arm Q fits on its 301,236 training records.

Label normalization: what it can and cannot move. The transformer is trained on the z-scored label $(y-\mu)/\sigma$. In the 010 configuration and arm R the two constants are the all-record statistics, $\mu = -0.008024324$ and $\sigma = 0.063263549$, where the training split alone gives -0.007688739 and 0.063186255 . Pearson and Spearman are invariant to an affine map of the labels, so no choice of μ and σ changes the correlation a given model would reach. What the constants set is the scale of the point loss relative to the other terms of Eq. 17: a different σ multiplies the standardized mean squared error by $(\sigma_{\text{all}}/\sigma_{\text{train}})^2 = 1.0024$, and a different μ is a constant offset the readout bias absorbs. Arm Q names its fit population, `transforms.fit_on: train`, and its constants are $\mu = -0.007843739$ and $\sigma = 0.063776418$; both sets are written by `experiments/025-solid-growth/scripts/label_normalization_constants.py`.

The loss functions differ, deliberately. The baselines minimize squared error. The transformer minimizes

$$\mathcal{L} = \lambda_p \mathcal{L}_{\text{MSE}} + \lambda_d S_\epsilon\left(\frac{1}{B} \sum_b \delta_{\hat{y}_b}, \frac{1}{B} \sum_b \delta_{y_b}\right) + \lambda_g \mathcal{L}_{\text{graph}}, \quad (17)$$

with $\lambda_p = 1$, $\lambda_d = 0.1$, $\lambda_g = 1$. The middle term is a debiased Sinkhorn divergence with $p = 2$ and blur 0.05 between the empirical measure of the batch’s predictions and that of the batch’s targets, computed unpaired, which shapes the marginal distribution of the predictions rather than their per-record accuracy. The third term is the graph penalty of Eq. 15. This is a difference in training objective, not in the evaluation, and it is part of what the transformer is. A distributional term can move Pearson and mean squared error in opposite directions, so the two metrics are read together rather than either alone.

Model size.

Table 6: Parameter budget. Two thirds of the transformer sits in the encoder, which does no per-sample work, and a quarter in the embedding table. The strain-dependent computation is 456,301 parameters, under a tenth of the model.

Component	Parameters	Role
Gene embedding table	1,189,260	the only place gene identity lives
Encoder, 8 layers	3,129,120	strain-independent, Section 4.2
Perturbation transform	391,140	the strain-dependent step
Readout head	65,161	pooling and MLP
CLS token	180	constant
Total	4,774,861	
B1 additive ridge	4,353	one scalar per gene, plus intercept
B5 embedding MLP	623,745	128 per gene, plus a two-layer network

5 The ladder under both splits ✗

Table 7: Test Pearson for every model under both arms, beside the 010 numbers. B0 through B4 are closed-form fits on a fixed split and carry no spread; B5 is the mean and standard deviation over three seeds. The transformer rows stand on different footings, named in the row label: the 010 checkpoints were scored on the 010 test split of 37,673 records; the GH 1598 and GH 1640 validation maxima are over 36 logged epochs; the last row is GH 1640’s validation-selected checkpoint scored on the arm Q test part.

Model	Arm R, random over records		Arm Q, query-pair disjoint
	010 build	025 build	025 build
B0 train mean	0.000	0.000	0.000
B4 query pair only	0.390	0.390	0.000
B3 screen mean	0.390	0.390	0.142
B1 additive per-gene ridge	0.400	0.400	0.185
B2 additive plus pair ridge	0.406	0.406	0.180
B5 embedding MLP, 3 seeds	0.426 $\pm$ 0.001	0.432 $\pm$ 0.005	0.141 $\pm$ 0.007
CGT, three 010 checkpoints, test	0.438 to 0.455		
CGT GH 1598, val max, epoch 14 of 36		0.446	
CGT GH 1640, val max, epoch 7 of 36			0.199
CGT GH 1640, epoch 7 checkpoint, test			0.139

5.1 Reading the ladder ✗

Arm R is the 010 result. Every closed-form model agrees with its 010 value to 3×10^{-13} , which follows from the record-level label and split identity of Section 2 rather than testing it. B5 differs by 0.006 against a three-seed standard deviation of 0.005; the 025 fit ran on CPU where 010’s ran on GPU, and the cause is untraced. GH 1598 reaches

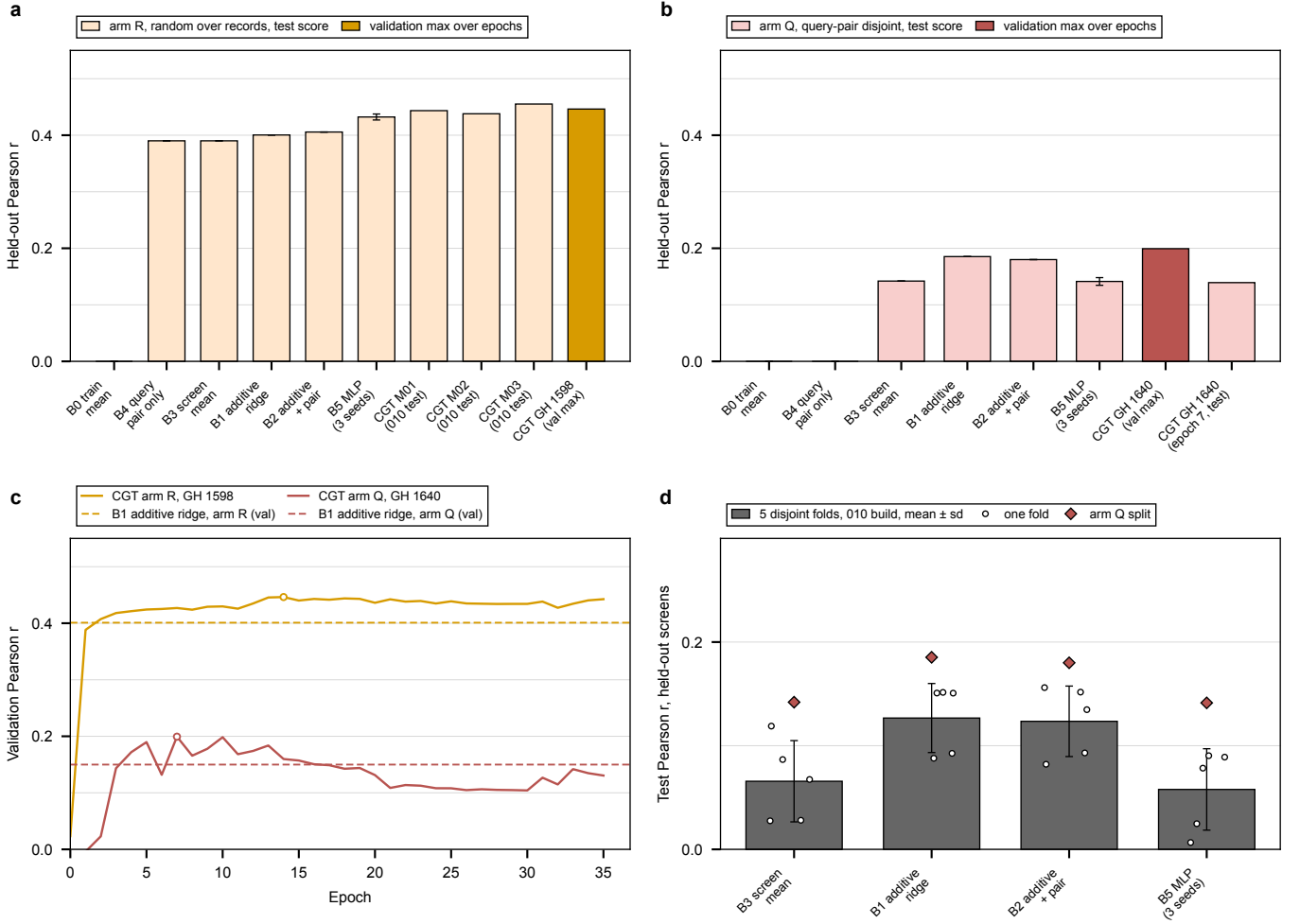


Figure 3: The ladder under both splits. **a**, arm R, random over records: the six baselines on the 37,673 test records, the three 010 checkpoints on the 010 test split, and the 025 replication GH 1598 as a validation maximum. **b**, arm Q, query-pair disjoint, on the 37,791 test records of 46 held-out screens, with GH 1640 as a validation maximum and its epoch 7 checkpoint scored on test. Within each arm, a light bar is a test score and a dark bar a validation maximum. Only B5 carries an error bar, the standard deviation over its three seeds; every other bar is one deterministic fit. **c**, validation Pearson per epoch for the two arms of the 010 configuration, each against the additive ridge fit on its own validation part; open circles mark each run's best epoch, the value the dark bars of a and b carry. **d**, how far the disjoint null moves with the choice of held-out screens: five disjoint folds on the 010 build per model, mean and standard deviation with each fold as a point, and the arm Q single split as a diamond. B0 and B4 are zero on every fold and are omitted.

a validation maximum of 0.446 at epoch 14, within 0.001 of the lower end of the 0.447 to 0.462 band the three 010 checkpoints recorded on validation.

The additive null is close, but the transformer does beat it. B1 reaches 0.400 against the transformer’s 0.438 to 0.455, so a model that cannot represent any interaction recovers 88 to 91 percent of the correlation. In explained variance the gap is $r^2 = 0.160$ against 0.207 for the best checkpoint, an increment of 0.047 of label variance. This is not the DeWitt result, where the network collapsed onto the additive model at $r > 0.999$. The margin is real, $+0.038$ to $+0.055$ with paired-bootstrap intervals that exclude zero (Section 5.4), and it is small.

Seed spread is a large fraction of that margin. The three transformer runs span 0.438 to 0.455, a standard deviation of 0.0088. The three B5 seeds span 0.425 to 0.427 on the 010 build, a standard deviation of 0.0011. So the transformer’s run-to-run spread is eight times the nonlinear baseline’s and roughly a fifth of its own margin over B1. A single checkpoint’s number is not the capability, and the three runs are not seed replicates in the strict sense: the training script fixes the seed at 42, M01 and M02 differ only in run-to-run nondeterminism, and M03 additionally changes the scheduler’s first cycle length.

Capacity closes about half the gap, and the graphs cannot be ruled out. B5 sees exactly B1’s features, has no graph and no attention, and reaches 0.426 on the 010 build and 0.432 on 025. That closes about half the distance from B1 to the transformer. Attributing the remainder to capacity alone does not hold, because the graphs shape the trained weights even though they are absent from the prediction function (Section 4.3), and Section 5.3 measures how heavily. What the ladder alone says is that richer set aggregation and eight times the parameters are worth at most 0.03 Pearson over B5, and that how much of that 0.03 is the graphs is not separable from these runs.

Spearman compresses the ladder further. On the 010 build the rank correlations run 0.406 for B1, 0.412 for B2 and 0.417 to 0.421 for B5, against 0.424 to 0.426 for the transformer. The gap from the additive null to the transformer is 0.020 in Spearman against 0.045 in Pearson. Ranking is what a design-space selection consumes, so the smaller of the two is the more decision-relevant number.

On the random split the ladder is mostly screen identity. B4 knows only which query pair a triple carries and never looks at the third gene. It reaches 0.390, 97 percent of the additive ridge and 86 percent of the best checkpoint. Those 420 pairs are the Kuzmin query doubles, the most frequent appearing 3,308 times, so B4 is close to a per-screen offset, and on a split that is random over records most of what every model in Table 7 is scoring is the identity of the screen a triple came from. It also explains why the additive null does so well on a label constructed to have the additive part removed: what B1’s per-gene coefficients absorb is largely query-pair structure reaching them through the genes that make up the query pairs.

Holding out screens collapses the ladder and inverts it. On arm Q the additive ridge falls from 0.400 to 0.185 and B4 to zero by construction. B5 falls from 0.432 to 0.141, below the ridge: the capacity that helped on unseen records of seen screens hurts on unseen screens. B3 falls back from a screen mean to a gene mean and lands at 0.142. The one transformer checkpoint with a test score on this split, GH 1640 at epoch 7, lands at 0.139, level with B5 and below the ridge (Section 6).

One disjoint split is a small sample of screens. Panel d is the reason arm Q’s numbers are read as one draw rather than as the disjoint null. Across five disjoint folds on the 010 build the ridge spans 0.088 to 0.151, 0.127 ± 0.033 , and arm Q’s 0.185 lies above every fold. The same holds for B2, B3 and B5, so arm Q is a comparatively easy draw of 46 held-out screens. A transformer number on arm Q is compared to the arm Q nulls, on the same 46 screens, and not to the fold mean.

Table 8: Held-out Pearson on the 010 build under the published random-over-records split and under splits that never share a Kuzmin query double. The single disjoint split is 010’s own (285, 67 and 68 query pairs), a different partition of the same 420 pairs from arm Q’s; the cross-validated column is the mean over five folds, each holding out a disjoint set of query pairs, with B5 additionally averaged over three seeds. B4 is structurally zero once query pairs are disjoint.

Model	Random split	Query-pair disjoint	
	test r	one split	5-fold mean $\pm$ sd
B0 train mean	0.000	0.000	0.000 ± 0.000
B4 query pair only	0.390	0.000	0.000 ± 0.000
B3 hierarchical mean	0.390	0.173	0.066 ± 0.039
B1 additive per-gene ridge	0.400	0.174	0.127 ± 0.033
B2 additive plus pair ridge	0.406	0.170	0.124 ± 0.034
B5 embedding MLP	0.426	0.150	0.058 ± 0.037

Under the disjoint folds the nonlinear baseline early-stops at epoch 3 to 7 rather than epoch 20 to 30, having reached training Pearson 0.43 while held-out performance was already falling. Generalizing to an unseen query double is a

different problem from generalizing to an unseen record of a seen query double, and the extra capacity that helped on the second hurts on the first.

5.2 Verifying the architecture reading ✗

Section 4 claims the encoder is strain-independent and that no adjacency tensor is an input to the prediction. Both are checkable rather than merely readable, and checking them also produces the per-record predictions that Section 5.4 needs. If the encoder really does not see the strain, then scoring a record needs only the perturbed gene indices, and the index space is the sorted base-graph node list, rebuildable from the genome without touching the LMDb. `experiments/010-kuzmin-tmi/scripts/score_010_checkpoints_directly.py` does that, loads each 010 checkpoint into the model class, and recomputes the metrics. The check is that the recomputed numbers must reproduce what the original evaluation runs recorded: a wrong gene ordering or a mis-loaded weight would give a plausible but wrong number.

Table 9: Scoring the 010 checkpoints through a hand-built forward pass that computes the encoder once and feeds it only perturbed-gene indices reproduces the recorded metrics. The residual difference is at the fourth decimal and is consistent with the original runs using mixed bf16 precision.

Checkpoint	Val Pearson		Val MSE	
	recomputed	recorded	recomputed	recorded
M01 <code>1zs9pcj3</code>	0.451970	0.451963	0.003244	0.003244
M02 <code>yv4r30bi</code>	0.447319	0.447155	0.003258	0.003259
M03 <code>c7671wgj</code>	0.461917	0.461881	0.003163	0.003163

Two details had to be right, and getting either wrong produced a plausible but wrong answer. The index space is the sorted genome gene set, 6,607 genes, not the build’s own 6,579 gene set; using the latter gave a validation Pearson of -0.001 to 0.002 . And the readout pools by mean, whereas the current class defaults to sum; using the default gave 0.420 to 0.438 , close enough to look credible and wrong. That reproduction is the direct verification of Section 4.2: the encoder was evaluated exactly once, on a strain-independent input, and the recorded per-split metrics came back. Setting the graph penalty weight from 1.0 to 0.0 on a trained checkpoint changes the maximum absolute test prediction by 9.0×10^{-7} , 1.2×10^{-6} and 1.3×10^{-6} , against a prediction spread of about 0.036 : adjacency is not an input to the prediction function. It says nothing about whether the graphs shaped the weights being scored.

5.3 What the graphs did during training ✗

Three independent measurements, none of which the inference-time ablation can see.

The penalty was almost the whole objective. The trainer logs the graph term, the point term and their normalized shares. Against a validation point loss of 0.81 the graph term is $5,740$, and the share `norm_weighted_graph_reg` is 0.99986 for all three checkpoints (Figure 2). Eq. 16 accounts for the size.

The regularized heads recover their graphs. The trainer scores each regularized head by asking, for every gene i with degree d_i , what fraction of i ’s true neighbors fall in that head’s top d_i attention targets.

Table 10: Neighbor recall at each gene’s own degree, layer 1, one head per graph. Ordinary self-attention has no reason to place its top d_i mass on a gene’s true neighbors, so these values are attributable to the penalty of Eq. 15.

Graph	Head	M01	M02	M03
physical	0	0.940	0.940	0.938
regulatory	1	0.992	0.992	0.991
tflink	2	0.979	0.979	0.979
string neighborhood	3	0.952	0.951	0.950
string fusion	4	0.999	1.000	1.000
string cooccurrence	5	1.000	1.000	1.000
string coexpression	6	0.729	0.731	0.729
string experimental	7	0.774	0.772	0.773
string database	8	0.996	0.996	0.995

Removing the penalty stops the model training. A matched pair of 30-epoch configurations exists for exactly this test, identical but for the graph term’s weight. With the weight at 1.0 , three replicates reach validation Pearson 0.464 , 0.456 and 0.456 . With it at 0.0 , the three companion runs reach -0.007 , -0.031 and a diverged run that returned no number, with training Pearson around 0.002 . That establishes that the graph term is load-bearing for optimization

in this configuration. It does not establish that the graphs supply biological information the model uses, because a term carrying 99.99 percent of the loss is also doing whatever conditioning, scaling and implicit regularization that share implies. Separating the two would need a control with a comparable penalty against permuted or degree-matched random graphs, which has not been run.

5.4 Record-by-record agreement $\times$

With per-record predictions available, the comparison can be made the way the DeWitt preprint makes it, between the predictions themselves rather than between their summary statistics. These are 010 checkpoints on the 010 test split, which is arm R’s test part record for record.

Table 11: Correlation between models’ test-set predictions, 37,673 records. The transformer does not reproduce the additive model, which is where 010 departs from the MULTI-evolve case. But two transformer training runs agree with each other at 0.788 to 0.821, barely more than a transformer agrees with the additive model at 0.726 to 0.759, and no more than it agrees with the nonlinear baseline at 0.784 to 0.810.

Model	Pearson with		
	CGT M01	CGT M02	CGT M03
B1 additive ridge	0.752	0.726	0.759
B2 additive plus pair	0.759	0.734	0.765
B4 query pair only	0.729	0.706	0.735
B5 embedding MLP	0.788	0.784	0.810
CGT M01		0.788	0.821
CGT M02	0.788		0.804

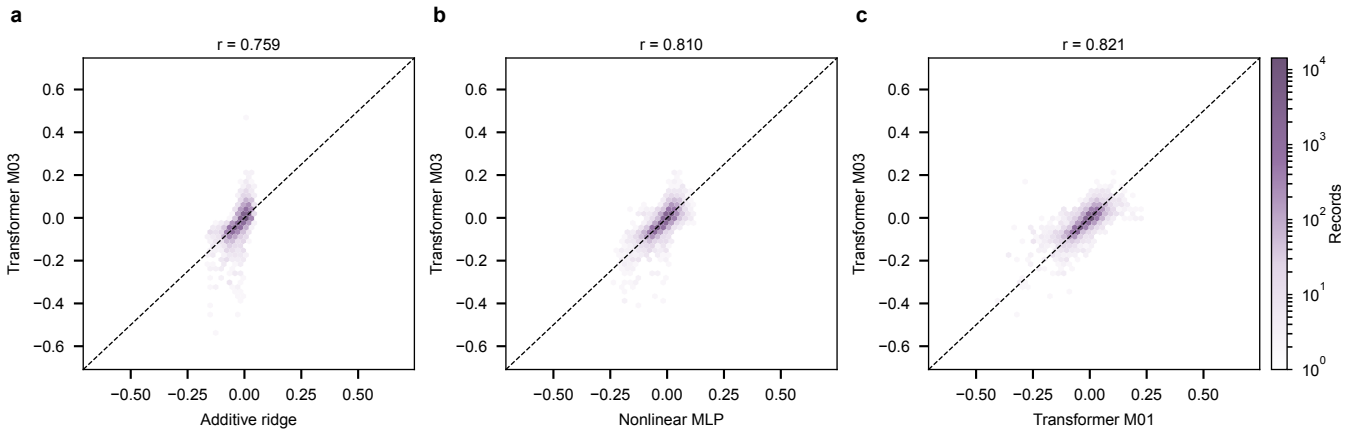


Figure 4: Test-set predictions of one model against another, 37,673 records as a log-density hexbin. **a**, the best transformer checkpoint against the additive ridge. **b**, against the nonlinear baseline on the same features. **c**, against a second transformer training run. The transformer is no closer to a second training run of itself than it is to a nonlinear baseline that has no graph and an eighth of the parameters.

The headline distinction from the preprint. DeWitt’s networks correlated with the additive model at $r > 0.999$, so the network *was* the additive model. Here the correlation is 0.73 to 0.76, so the transformer is computing something the additive model does not.

How reproducible the extra content is. Regressing each model’s predictions on B1’s and correlating what is left isolates the non-additive component. It is 43.5, 47.3 and 42.4 percent of each checkpoint’s prediction variance. Between two checkpoints it correlates at 0.533, 0.584 and 0.564. Against the part of the label B1 also fails to explain it correlates at 0.236, 0.234 and 0.253, where B5’s non-additive component manages 0.165. So the non-additive content is neither noise nor stable: about half of it is shared between two training runs, and the shared part carries real signal about the residual label.

The models are wrong in the same places. Correlating the residuals $y - \hat{y}$ of B1 and each checkpoint gives 0.907, 0.897 and 0.918. Almost all of the error is shared, so the transformer is not fixing a class of records the additive model gets wrong; it is making a modest, broadly distributed improvement.

The advantage is real and bounded. A paired bootstrap over test records, 2,000 resamples, puts the Pearson gain over B1 at +0.043 with a 95 percent interval [+0.023, +0.063] for M01, +0.038 with [+0.019, +0.058] for M02, and +0.055 with [+0.034, +0.074] for M03. All three intervals exclude zero. B5 gains +0.025 with [+0.017, +0.033].

Rankings diverge where selection happens. Among the 100 records each model calls most negative, B1 and M03 share 31, B5 and M03 share 45, and B1 and B5 share 46. At $K = 10,000$ all pairs exceed 0.91. Between two transformer checkpoints the overlap rises monotonically with K : 0.10 to 0.30 at $K = 10$, 0.39 to 0.47 at $K = 100$, 0.57 to 0.60 at $K = 1,000$, and 0.91 at $K = 10,000$. An extreme quantile is where the fitted surface is least constrained by data and where the label itself is noisiest, so two runs that agree about the bulk should be expected to disagree about the last hundred; the consequence for practice is a reason to ensemble.

Prediction spread. Test prediction standard deviations are 0.025 for B1, 0.029 for B5 and 0.036 to 0.038 for the checkpoints, against a test label standard deviation of 0.064. Every model is heavily shrunk toward the mean, as a squared-error fit on a noisy target should be.

6 The transformer on held-out screens ✗

6.1 The run registry ✗

Every transformer number in this document traces to one of eight training runs. Table 12 names each once, with the configuration it ran and what that configuration turns on, and links its W&B page, which holds the per-epoch training loss, validation Pearson and the selected epoch. The table is generated from the W&B API and no run id is typed into the text.

Table 12: The run registry. The 010 checkpoints are arm R’s reference and were scored on the 010 test split by their own evaluation runs. GH 1598 and GH 1640 were stopped by the 12 hour wall clock and have no test evaluation; GH 1640’s epoch 7 checkpoint was scored afterward on CPU. The IGB runs finished 30 epochs with no test evaluation. The validation maximum is an upward-biased order statistic over the logged epochs. Configs of the IGB runs are on branch `feat/025-fitness-joint-head`.

Run	W&B	Arm	Config	Gene input	Schedule, readout	Epochs	Val max (ep.)	Test
CGT M01	1zs9pcj3	R (010 split)	<code>cabbi_002</code>	learnable table	cosine, 30-epoch first cycle, mean	64	0.452 (24)	yes, 010 eval run
CGT M02	yv4r30bi	R (010 split)	<code>cabbi_002</code>	learnable table	cosine, 30-epoch first cycle, mean	64	0.447 (25)	yes, 010 eval run
CGT M03	c7671wgj	R (010 split)	<code>cabbi_002</code>	learnable table	cosine, longer first cycle, mean	49	0.462 (24)	yes, 010 eval run
GH 1598	0yw7moue	R	<code>s0_r_kl_000</code>	learnable table	cosine, 010 schedule, sum	36	0.446 (14)	no
GH 1640	327csnlk	Q	<code>s0_q_kl_004</code>	learnable table	cosine, 010 schedule, sum	36	0.199 (7)	epoch 7 checkpoint, CPU
IGB 2391132	s1vx2zgw	Q	<code>s0_q_kl_emb_017</code>	four-region sequence composite	constant 2.5e-4, 30 epochs, perturbed CLS	30	0.262 (2)	no
IGB 2391133	8aa08xx0	Q	<code>s0_q_kl_calm_020</code>	CaLM codon embedding	constant 2.5e-4, 30 epochs, perturbed CLS	30	0.252 (7)	no
IGB 2391134	pmkzwwzw	Q	<code>s0_q_kl_prot_021</code>	ProtT5 protein embedding	constant 2.5e-4, 30 epochs, perturbed CLS	30	0.270 (3)	no

What each configuration turns on. The arm Q runs differ from one another in three settings and share everything else. GH 1640 is the 010 configuration on the disjoint split: the learnable gene table, the cosine schedule with a 30 epoch first cycle, sum pooling in the readout, and the KL penalty on layer-1 attention with the corrected edge normalization. The three IGB runs come from the joint-fitness-head branch (PR #346, not yet on `main`): they replace the learnable table with a fixed sequence embedding of matched parameter count, read out through a perturbed CLS token, and train at a constant learning rate of 2.5×10^{-4} for 30 epochs. They share the split and the graph penalty with GH 1640 and nothing else a replicate would have to share, so they are four runs on one split, not one run replicated.

The control that is missing. The IGB runs change the gene input together with the readout and the schedule, so a difference between any of them and GH 1640 has three candidate explanations. The arm that separates them is the learnable table under the IGB schedule and readout, `cgt_s0_q_kl_ctrl_016`. Its config exists on the branch and it had no run on W&B as of 2026-09-15. Until it runs, the embedding variants are validation trajectories beside GH 1640 and not a comparison with it.

6.2 Validation, then the one test score ✗

Every run peaks early and falls. GH 1640 passes the additive validation null of 0.150 in epoch 4, peaks at 0.199 in epoch 7, is last above the null in epoch 16 and ends at 0.131, below every null but B3. The three IGB runs peak higher, 0.252 to 0.270 in epochs 2 to 7, and decline from there. The composite and CaLM embeddings end at 0.222

Table 13: The four arm Q runs by their validation trajectory. Epochs is the number logged.

Run	Config	Gene input	Schedule, readout	Epochs	Val max (epoch)	Val last
GH 1640	cgt_s0_q_kl_004	learnable table	cosine, 010 schedule, mean	36	0.199 (7)	0.131
IGB 2391132	cgt_s0_q_kl_emb_017	four-region sequence composite	constant 2.5×10^{-4} , perturbed CLS	30	0.262 (2)	0.222
IGB 2391133	cgt_s0_q_kl_cal_m_020	CaLM codon embedding	constant 2.5×10^{-4} , perturbed CLS	30	0.252 (7)	0.208
IGB 2391134	cgt_s0_q_kl_prot_021	ProtT5 protein embedding	constant 2.5×10^{-4} , perturbed CLS	30	0.270 (3)	0.135

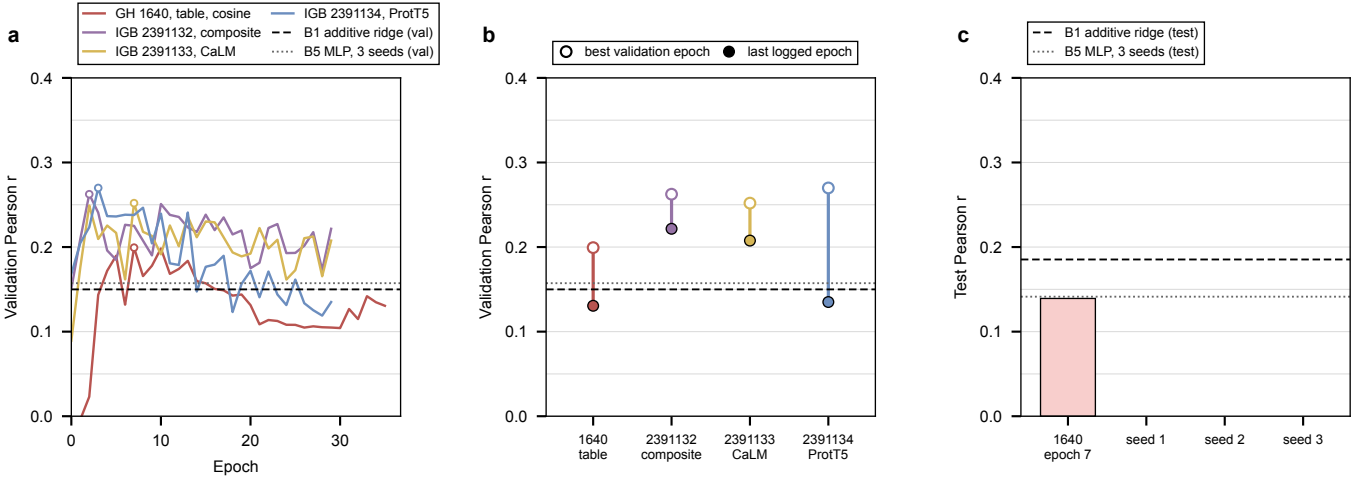


Figure 5: Every transformer run on the query-pair-disjoint split. **a**, validation Pearson per epoch on the 37,705 validation records of 43 held-out screens, with the additive ridge and the three-seed MLP fit on the same validation part as horizontal lines; open circles mark each run’s best epoch. **b**, the same runs reduced to their best epoch (open) and last logged epoch (filled). **c**, the test comparison on the 37,791 test records of 46 held-out screens: the one scored checkpoint, GH 1640 at its validation-selected epoch 7, against the arm Q test nulls. The three empty slots are the seeds of the 010 configuration that have not been run, each to be scored at its best validation epoch.

and 0.208 and stay above the ridge through their last five epochs; ProtT5 ends at 0.135, level with GH 1640. Arm R has the opposite shape, a margin over its null that appears in epoch 2 and holds for the rest of the run (Figure 3c).

The epoch 7 checkpoint on test. The checkpoint GH 1640 saved at its best validation epoch was scored on CPU on the arm Q test part (`experiments/025-solid-growth/scripts/score_cgt_checkpoint_cpu.py`, GilaHyper job 1846). The scorer reproduces the run’s logged validation Pearson at that epoch, 0.1997 recomputed against 0.1993 logged, the fp32-against-bf16 gap the 010 reproduction also showed, and two details had to be right for it to: the index space is the genome’s 6,607 genes, and this checkpoint pools by sum where the 010 checkpoints pooled by mean. On test the same checkpoint reaches 0.139 Pearson and 0.109 Spearman.

Table 14: GH 1640’s epoch 7 checkpoint against three nulls on the arm Q test part, 37,791 records. Gain is the transformer’s Pearson minus the baseline’s; the interval is a paired bootstrap over records, 2,000 resamples, percentile method. Residual correlation is between $y - \hat{y}$ of the two models; prediction correlation is between the two prediction vectors.

Against	Baseline r	Gain	95% interval	Residual corr.	Prediction corr.
B1 additive ridge	0.185	−0.046	[−0.056, −0.036]	0.944	0.637
B2 additive plus pair	0.180	−0.041	[−0.051, −0.030]	0.943	0.642
B5 embedding MLP, seed 0	0.138	+0.001	[−0.008, +0.010]	0.960	0.752

The checkpoint sits 0.046 below the additive ridge, with a paired interval that excludes zero, and 0.041 below the ridge with pair offsets. Against the nonlinear MLP the gain is 0.001 with an interval that straddles zero: on held-out screens the transformer and the MLP on the same features are indistinguishable, and both are below additivity. The residual correlations of 0.94 to 0.96 say the models are wrong on the same records; the prediction correlation with B1 of 0.64 is well below the 0.73 to 0.76 the 010 checkpoints showed on the random split, so the transformer departs further from the additive model on unseen screens and loses accuracy in doing so.

6.3 The prespecified decision ✗

A transformer result on arm Q is a claim only under three criteria, fixed before the scoring ran. D1 asks whether the test Pearson at the validation-selected epoch exceeds the additive ridge’s. D2 asks whether the paired-bootstrap interval of the difference excludes zero, so that a pass on D1 is more than one draw of records. D3 asks whether D1 and D2 hold over three seeds of the configuration, so that a pass is more than one draw of the optimizer.

Table 15: The decision on the evidence available. D3 has not been run and the verdict is provisional on it.

Criterion	Statement	Value	Result
D1	transformer test Pearson at its validation-selected epoch exceeds B1’s	0.139 against 0.185	FAIL
D2	paired-bootstrap 95 percent interval of transformer minus B1 excludes zero	-0.046, [-0.056, -0.036]	FAIL
D3	D1 and D2 hold over three seeds of the configuration	one run, one checkpoint	NOT RUN

Verdict, on one checkpoint: the transformer in the 010 configuration is not shown to beat the additive null on held-out screens, and the one scored checkpoint is below it. D1 and D2 fail in the direction of the null, so the conclusion is not an absence of evidence: on these 46 screens the additive ridge is the better predictor by 0.046 Pearson with an interval that excludes zero. What the verdict does not cover is the configuration’s capability, which one run and one checkpoint do not measure; D3 is what turns this into a statement about the model rather than about GH 1640.

Hypothesis, untested. The decline after the peak in every arm Q run is consistent with the model fitting screen-specific structure of the training pairs that does not transfer, the quantity B4 measures. No measurement here separates that from ordinary overfitting of label noise, and no mechanism is claimed.

6.4 Corrections recorded ✗

Three claims made earlier in this analysis were wrong and are corrected here rather than silently folded in. The Delta package for the graph-regularization sweep carried 010’s own query-pair-disjoint split, 285, 67 and 68 pairs, rather than arm Q’s, 331, 43 and 46; a result on it would have compared to neither the arm Q nulls nor GH 1640, and the index artifact now carries arm Q’s partition by query pair name and asserts the fold sizes reproduce. The first version of this document stated GH 1640 first cleared its validation null in epoch 3 and that GH 1598’s validation maximum lay inside the 010 checkpoints’ band; the history gives epoch 4, and 0.446 lies 0.001 below the band. And the placeholder for the test comparison described GH 1640’s checkpoint as unscorable without a GPU; the encoder runs once and the per-record work is one cross-attention, so it scored in 13 minutes on 32 CPUs.

7 What is established, and what is left ✗

7.1 Established ✗

The additive null is fit on the same records, splits and label the transformer uses, with the ridge penalty chosen on validation and reported on test, under both arms. On the published split the transformer’s margin over a model that cannot represent interaction is real and small, and most of what every model scores there is screen identity: a model that never looks at the third gene reaches 97 percent of the additive null, and holding out whole query doubles drops the additive null from 0.400 to 0.185 on arm Q and to 0.127 ± 0.033 over five folds, and inverts the ordering of the baselines. Every model in the comparison receives the same information, the identity of the three perturbed genes; that is a reading of the code, verified by a hand-built forward pass that reproduces the recorded metrics. The graphs are load-bearing during training: they carried 99.99 percent of the 010 loss, their heads recover their graphs, and the matched pair of runs without the penalty does not train.

7.2 Not established ✗

The transformer’s capability on held-out screens is not measured. One checkpoint of one run has a test score on arm Q, and it is below the additive null with an interval that excludes zero (Table 15). That is a measurement of GH 1640’s epoch 7 checkpoint. Whether the configuration can do better, or whether a seed-averaged run lands where this one did, is D3, and it has not been run. Whether the +0.04 margin of the random split survives held-out screens is therefore answered for one draw of the optimizer and open for the model.

The reproduction is faithful but not bit-exact. The hand-built forward pass matches the recorded 010 metrics to the fourth decimal, not exactly. The original runs used mixed bf16 precision and the reproduction runs in fp32, which is the likely source and has not been verified by rerunning in bf16. The conclusions of Section 5.4 rest on correlations at the second decimal and are not sensitive to a fourth-decimal shift.

One seed of the nonlinear baseline enters the paired analysis. Table 11 uses B5 seed 0. The three seeds agree closely on aggregate metrics, but their record-level agreement with each other was not measured.

Whether the graphs carry information is separate from whether they are load-bearing. The runs without the graph penalty fail to train, and the penalty was almost the entire loss. Neither separates the graphs supplying biological structure from a large auxiliary term supplying conditioning that the optimization needs. The control that would separate them, a comparable penalty against degree-matched random graphs, has not been run, and the matched pair itself ran 30 epochs with one diverged arm rather than three replicates.

Nothing here bears on the architecture’s merits elsewhere. The set-function reduction of Section 3.9 is specific to the 010 configuration, where the masking and propagation arms are off and the perturbation transform is one block. A configuration that switches those on would have a strain-dependent encoder path and would need its own reading.

7.3 What is left before this enters the main text ✗

1. **Replicate the 010 configuration on arm Q.** Three seeds of `cgt_s0_q_k1_004` with a test evaluation at the best validation epoch, under one fixed epoch budget. GH 1640 peaked at epoch 7 and 36 epochs cost the 12 hour wall clock, so a 20 epoch budget with checkpoint-on-best is enough. These fill the empty slots of Figure 5c, decide D3 of Table 15, and give the transformer row of Table 7 a spread. The CPU scorer and the paired bootstrap run unchanged on each.
2. **Run the missing control.** `cgt_s0_q_k1_ctrl_016`, the learnable table under the IGB schedule and readout, with seeds and test scores; without it the embedding variants of Table 12 stay validation trajectories and do not enter Figure 3.
3. **Fold spread for the transformer, or an explicit statement that it is unaffordable.** Panel d of Figure 3 is five folds for the nulls. A transformer fold costs about 12 hours of one GilaHyper node per 36 epochs, so five folds by three seeds is 15 such runs. If the main text reports arm Q alone it should say so and cite the fold spread of the null as the scale of the split-to-split variation.
4. **B5 on GPU.** The 025 B5 fit ran on CPU and differs from 010’s GPU fit by 0.006 on arm R. A GPU refit closes the one row of Table 7 that is not an exact reproduction.
5. **The graph control.** A run with the penalty of Eq. 15 against degree-matched random graphs, to separate the graphs carrying structure from a large penalty carrying conditioning.

6. **Bibliography.** The Visani preprint, Tran 2026 and Kuzmin 2018 are named by DOI in prose. A Zotero collection for this document, then `make bib-pull`, makes them citations.

What is not carried over from the 010 report. The gene-index defect of the 010 inference runs, a uniform 28-position shift that invalidated the design-space selections, is in `notes-tex/010-index-defect` and the 010 report's design-space section, and bears on the panel selection rather than on the ladder.